AI Engineer — Take-Home Task

Build and evaluate a policy-grounded insurance claims review agent

Purpose We want to understand how you use AI to build a working agentic application: how you decompose the problem, use models and tools, ground decisions in evidence, test behaviour, and make pragmatic engineering trade-offs.

Overview

You will build a small but working AI agent for a health insurance claims workflow. The emphasis is on an end-to-end prototype that we can run and discuss with you — not on production polish or a large codebase.

Item	Expectation
Expected time	Approximately 4–6 hours. Please time-box the exercise rather than over-engineering it.
Primary deliverable	A runnable repository or zipped project with a short README and instructions.
Supporting material	A concise architecture note (1–3 pages or equivalent README section) and evaluation results.
Interview	Be prepared to demo the agent and discuss your design, prompts, evaluations, failure modes, and next steps.
AI tools	Use coding assistants, LLMs, frameworks, or agent libraries as you normally would. Tell us what you used and where it helped.

The scenario

You are building a claims review agent for a health insurer. For each claim, the agent receives structured claim data and must consult the relevant insurer policy document before deciding the next action. The agent must remain grounded in the supplied policy and claim evidence and must not invent business rules.

Allowed actions

- approve_pre_authorization()
- approve_claim_payment(payment_in_dollars: float)
- deny_claim()
- request_missing_documentation(documents: list[str])
- route_to_senior_reviewer()
- route_to_coding_review()

Core workflow rules

- **Inactive plan:** if member_status is inactive, deny the claim.
- **Missing essential documentation:** request the missing information rather than making an unsupported decision.
- **High-value claim:** if claimed_amount is greater than $5,000, route to a senior reviewer.
- **Diagnosis / procedure mismatch:** route to coding review and do not approve.
- **Pre-approval:** validate policy coverage, medical-necessity evidence, diagnosis and required documentation before approval or denial.
- **Appeal:** consider the prior denial reason and whether new evidence is relevant and sufficient.

Your task

Build an MVP that takes a claim as input, retrieves the appropriate policy evidence, follows the workflow rules, and returns a structured decision. You may choose the language, model provider, orchestration approach, retrieval stack, and interface.

1. Build the agent

1. **Claim understanding.** Parse the structured claim, identify claim type and relevant state, and determine which workflow applies.
2. **Policy retrieval.** Load and search unstructured policy PDFs. Ensure the claim is evaluated only against the policy for its insurance provider.
3. **Reasoning and control flow.** Combine deterministic rules and model reasoning in a way that is inspectable and robust. We are interested in where you do — and do not — let the LLM decide.
4. **Tool/action execution.** Select one of the allowed actions, validate its arguments, and record enough information to understand why it was selected.
5. **Grounded response.** Return the decision, concise reason, and the policy/claim evidence used. Do not invent missing facts.

2. Make it runnable

- Provide a simple interface: CLI, API endpoint, notebook, or lightweight UI are all acceptable.
- Include setup and run instructions. If an external model key is required, explain the expected environment variable and provide a graceful fallback or mock where reasonable.
- Include at least a few example runs covering a normal path and one or more edge cases.
- Log or expose enough intermediate state that we can inspect retrieval, selected evidence, decision, and tool call.

3. Evaluate it

- Create a small evaluation set that covers multiple claim types and important overrides.
- Measure at minimum: action correctness, evidence grounding, and robustness to missing or ambiguous information.
- Include at least one adversarial or failure-oriented test, such as irrelevant policy text, conflicting evidence, or ambiguous provider notes.
- Summarise results and identify the highest-risk failure mode you would address next.

The exercise materials should include the supplied claim examples and policy PDF(s). You may add clearly-labelled synthetic cases for evaluation.

What to submit

- **Code.** A runnable repository or zip containing the application and tests/evaluation harness.

- **README.** Setup, run instructions, architecture overview, key assumptions, and the AI/coding tools you used.
- **Evaluation evidence.** A compact table or report showing test cases, expected vs. actual outcome, and notable failures.
- **Optional demo.** A short recorded walkthrough is welcome but not required.

Technical freedom

You do not need to reproduce a specific stack. We care more about the quality of the engineering decisions than whether you use a particular framework. A small, well-structured implementation with clear boundaries is preferable to a large application with hidden behaviour.

Assumptions you may make

- The supplied policy PDFs are the source of truth for coverage rules.
- The structured claim object is trusted input, although fields may be missing or ambiguous.
- You may create additional synthetic claims for evaluation, but clearly label them as synthetic.
- You may stub external systems; the task does not require real payment, CRM, or insurer integrations.
- You may use public libraries and public model APIs. Do not use or upload confidential data outside the materials provided for the exercise.

What we are looking for

Area	What good looks like
Agent engineering	A working end-to-end agent with sensible decomposition, state handling, tool use, and failure behaviour.
Grounding & retrieval	Relevant policy evidence is selected reliably and decisions can be traced back to it.
Use of AI	Effective use of models and AI-assisted development, with judgement about what should remain deterministic.
Evaluation mindset	Tests go beyond happy paths and help explain whether the system is safe enough to improve.
Software quality	Readable code, clear interfaces, reproducibility, and pragmatic scope.

Communication	You can explain trade-offs, limitations, and what you would do next in a production setting.

Important We are not testing how much code you can write. We are testing whether you can use AI and software engineering together to build a reliable agentic application.

Discussion prompts for the interview

- Where could the model make an unsafe or unsupported decision, and how did you constrain that?
- What did your evaluations reveal that changed your implementation?
- What would you change for production scale, observability, cost, latency, and compliance?
- Which parts of your solution were generated or accelerated with AI tools, and how did you validate them?